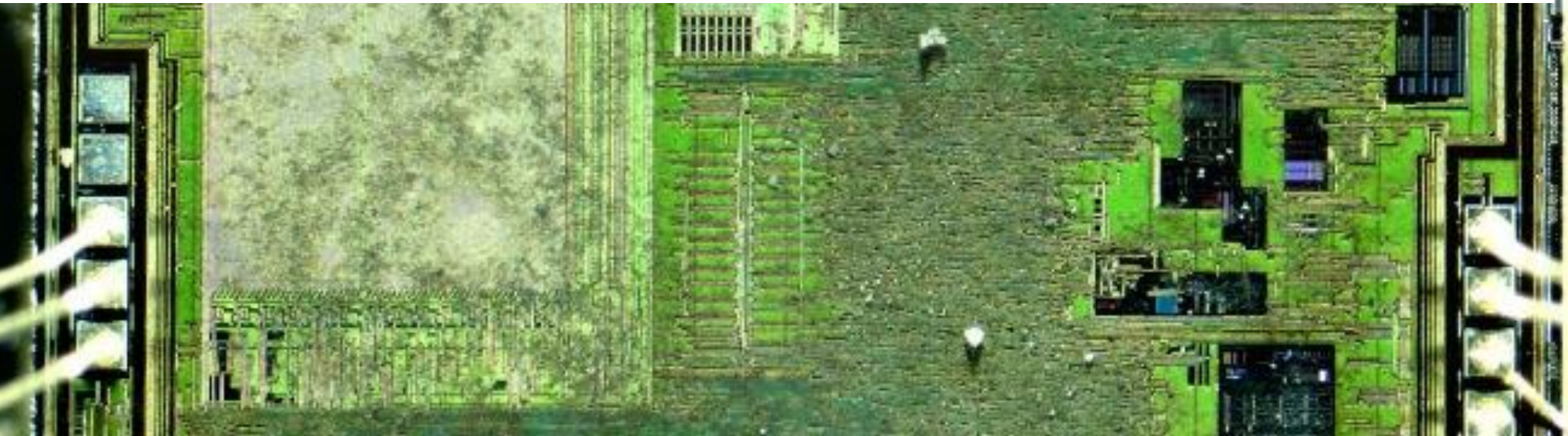
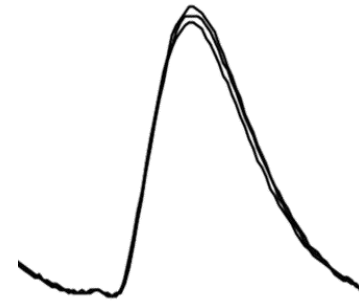
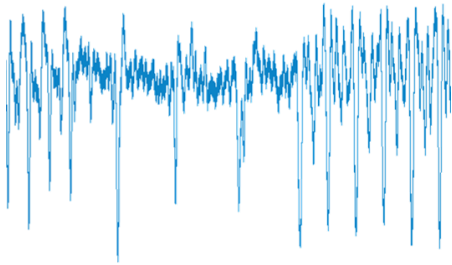


Code:

Differential Power Analysis

David Oswald



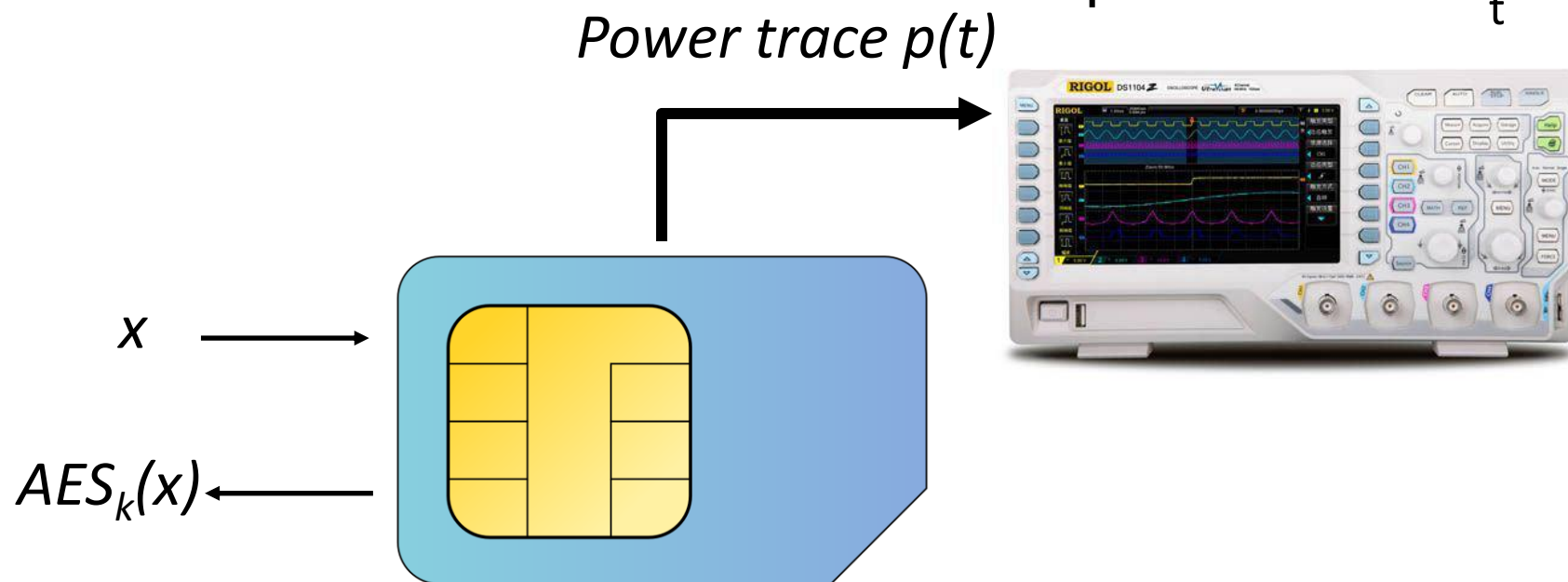
Goals of this lecture

After this lecture, you will ...

- Understand differential power analysis
- See how even small leakages can lead to very efficient attacks
- Have an idea about the need for and cost of countermeasures

Attack setup

3



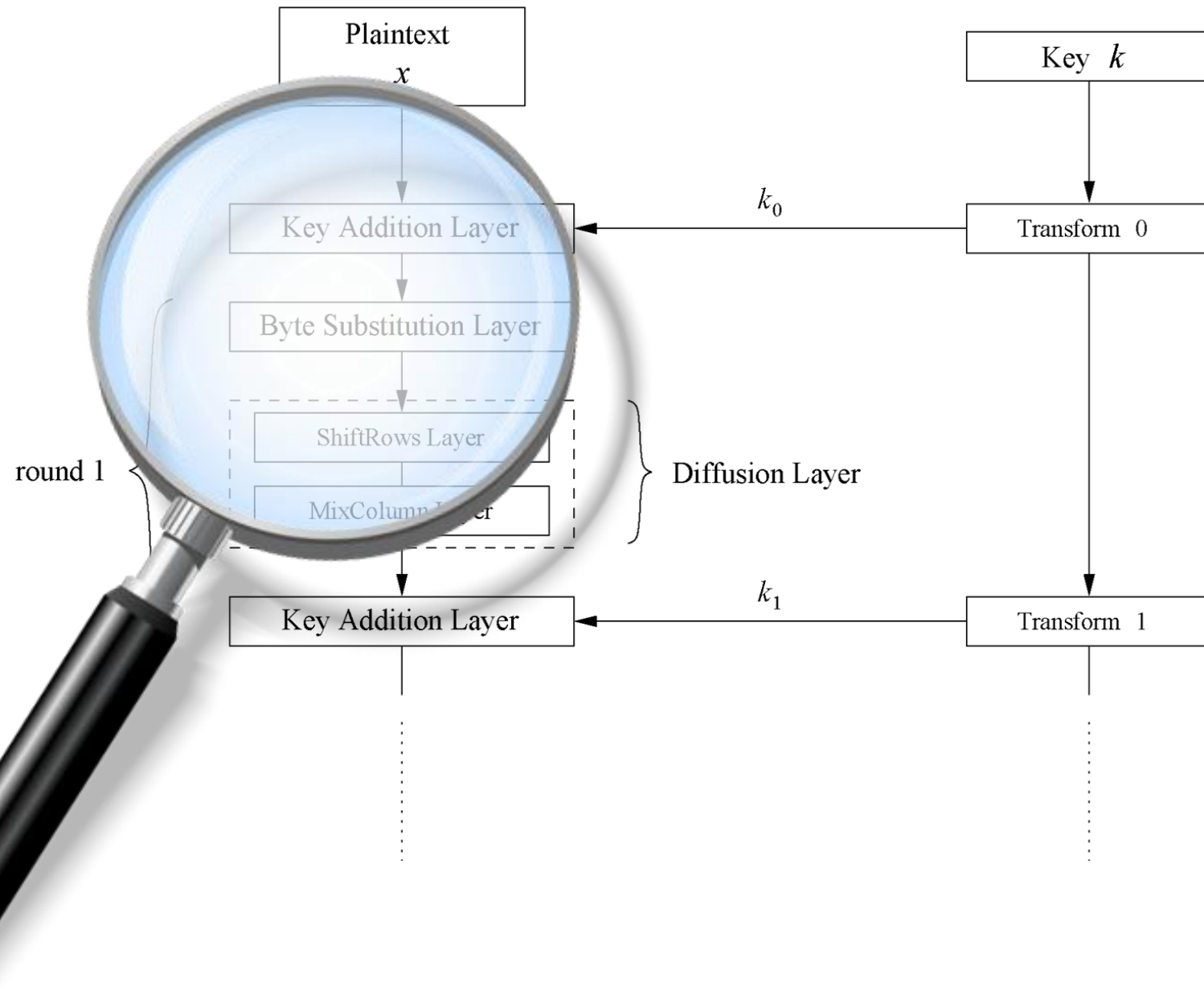
We repeat this for N different x (N large!)

$x^{(0)} \dots x^{(N-1)}$

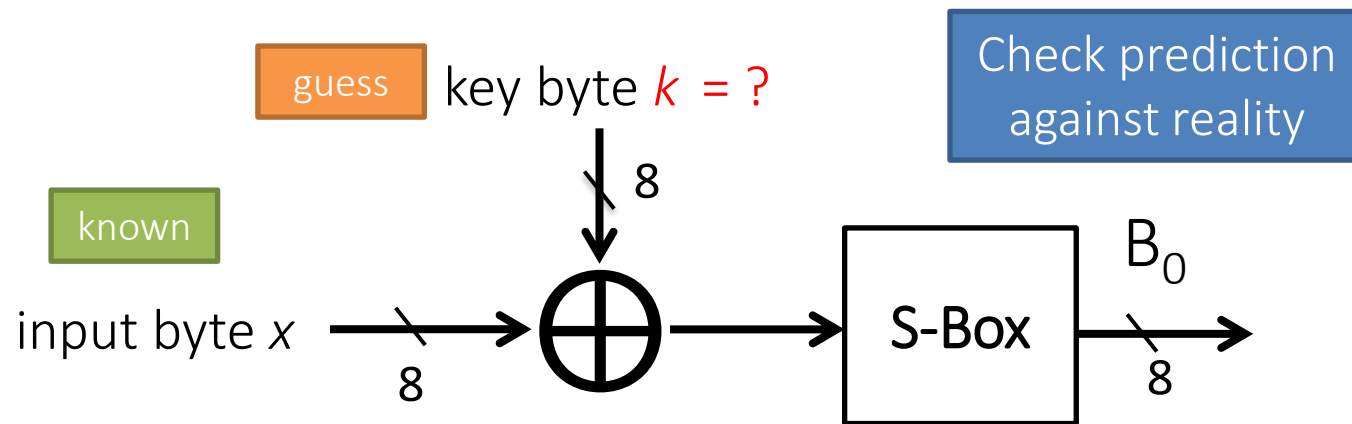
and store the respective power traces

$p^{(0)}(t) \dots p^{(N-1)}(t)$

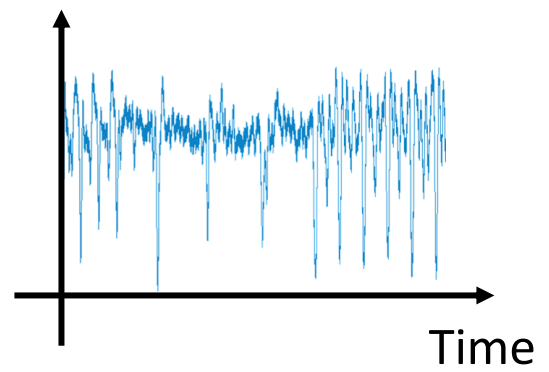
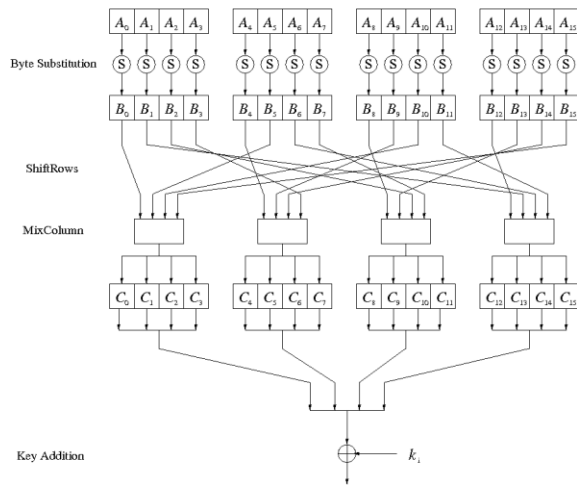
AES: overview



Let's look at a single byte



$$B_0 = \text{sbox}[x[0] \wedge k[0]]$$



Differential Power Analysis on AES



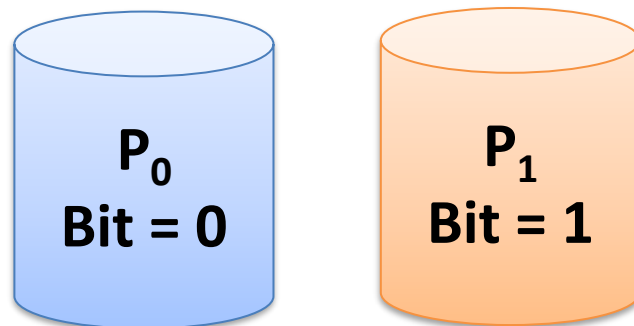
The S-Box lookup

$$B_0 = \text{sbox}[x[0] \wedge k[0]]$$

- The S-Box is constant-time (if no cache used)
- But the power consumption at the correct clock cycle depends on the output B_0
- If we measure this power consumption, can we **recover** $k[0]$?

The idea ...

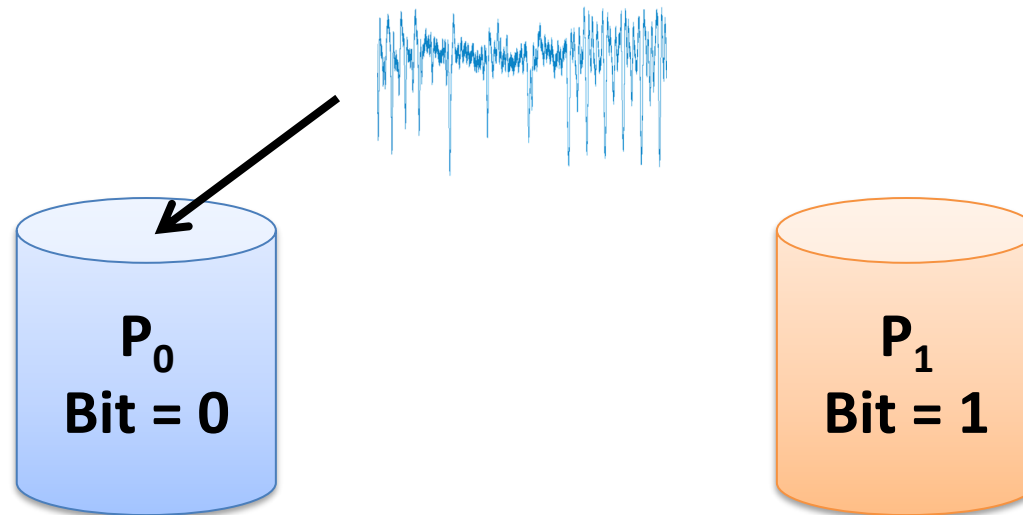
- Similar to the timing attack, we
 - Focus on a single key byte
 - Go over all 256 candidates for that byte
 - Go over many $(x, p(t))$ pairs
- For each key candidate k , we compute (for each input x) if $\text{MSBit}(B_0) = 1$ or $= 0$
- We group the measured power trace into two “heaps”



Putting traces into two groups

Example: Guess for first key byte: $k = 0x00$

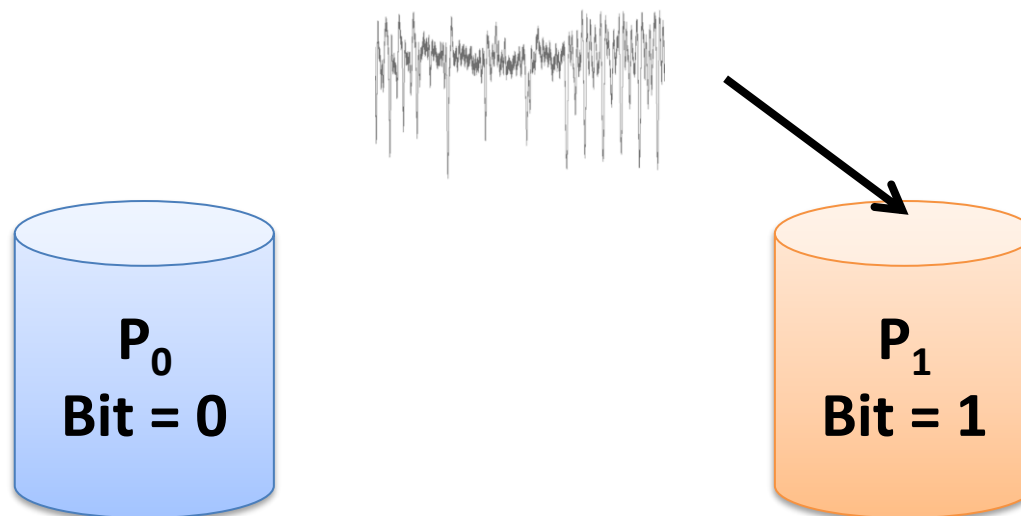
Trace	0	1	2	3	4	5	...
Plaintext x	0x70						...
Output S-Box B_0	0x51						...
MSBit(B_0)	0						...



Putting traces into two groups

Example: Guess for first key byte: $k = 0x00$

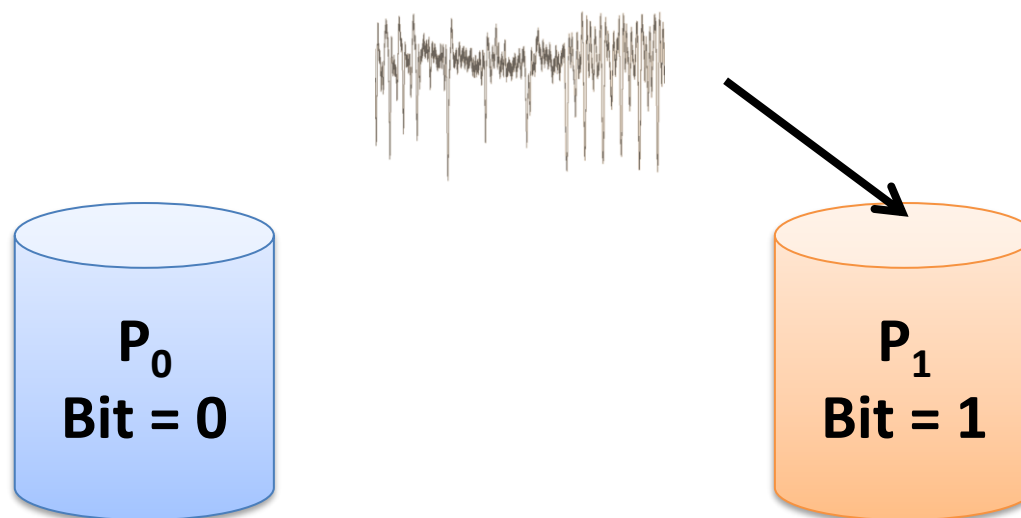
Trace	0	1	2	3	4	5	...
Plaintext x	0x70	0x22					...
Output S-Box B_0	0x51	0x93					...
MSBit(B_0)	0	1					...



Putting traces into two groups

Example: Guess for first key byte: $k = 0x00$

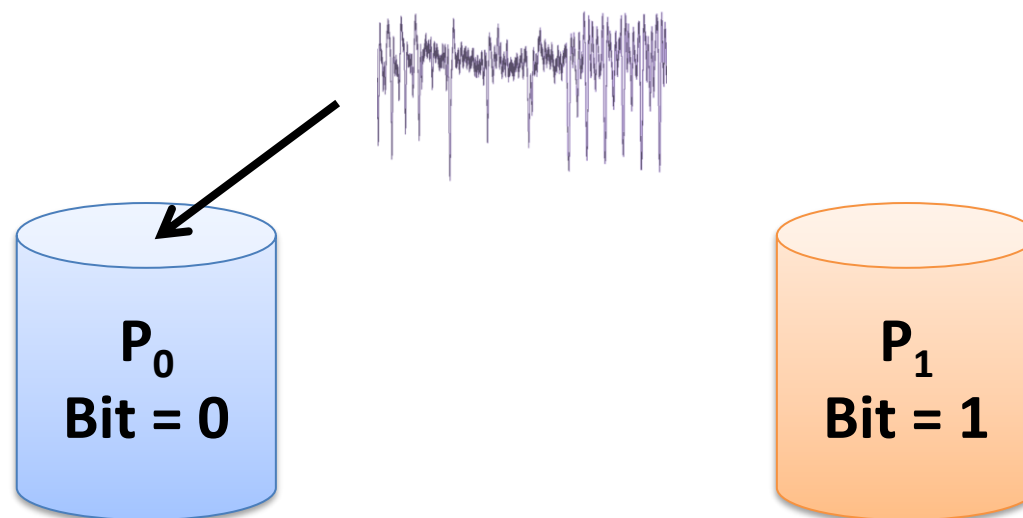
Trace	0	1	2	3	4	5	...
Plaintext x	0x70	0x22	0x0C				...
Output S-Box B_0	0x51	0x93	0xFE				...
MSBit(B_0)	0	1	1				...



Putting traces into two groups

Example: Guess for first key byte: $k = 0x00$

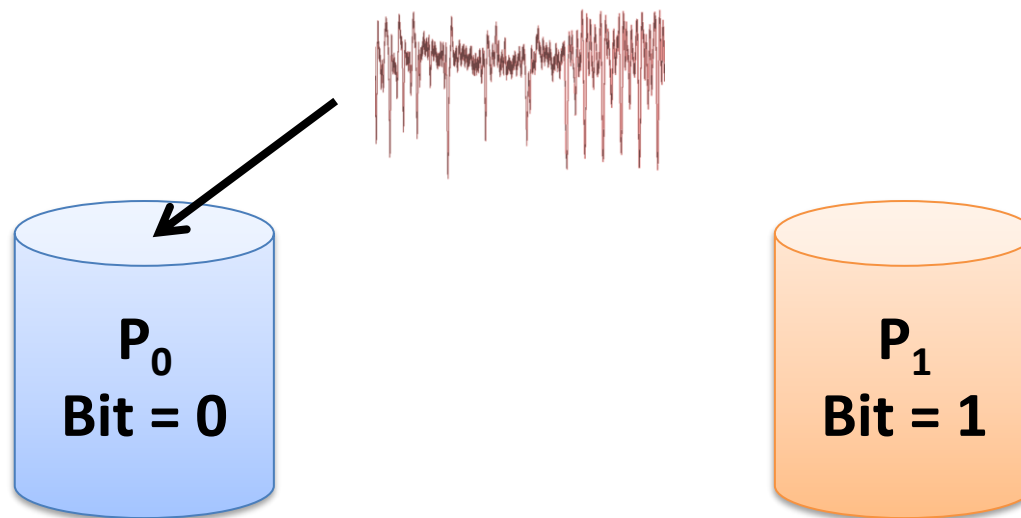
Trace	0	1	2	3	4	5	...
Plaintext x	0x70	0x22	0x0C	0xFF			...
Output S-Box B_0	0x51	0x93	0xFE	0x16			...
MSBit(B_0)	0	1	1	0			...



Putting timing into two groups

Example: Guess for first key byte: $k = 0x00$

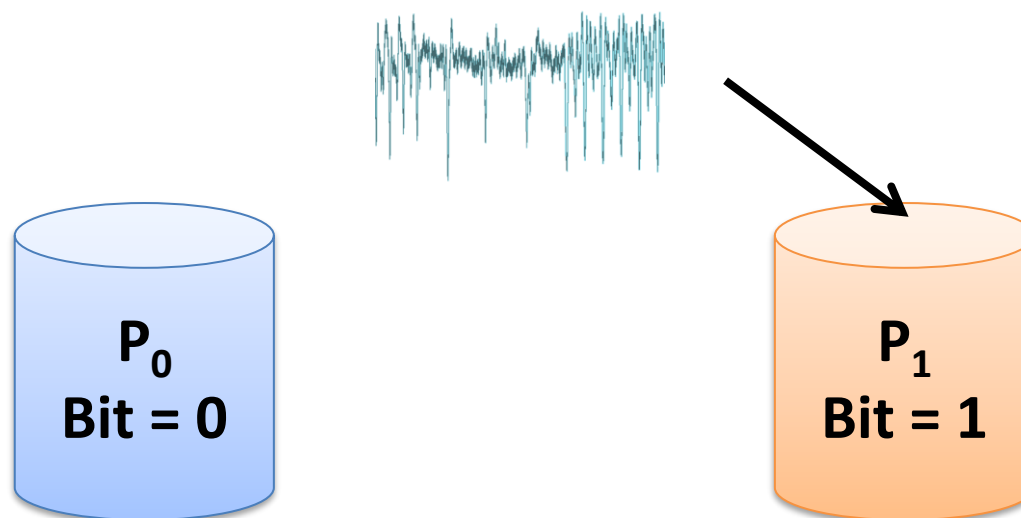
Trace	0	1	2	3	4	5	...
Plaintext x	0x70	0x22	0x0C	0xFF	0x01		...
Output S-Box B_0	0x51	0x93	0xFE	0x16	0x7C		...
MSBit(B_0)	0	1	1	0	0		...

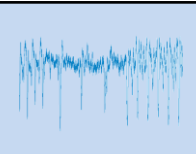
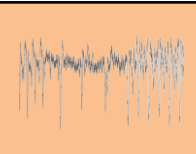
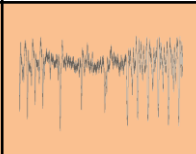
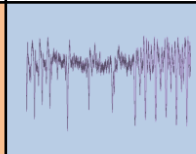
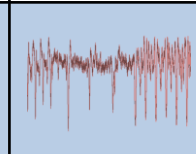
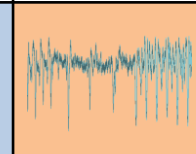
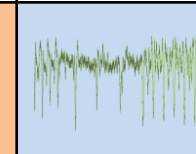
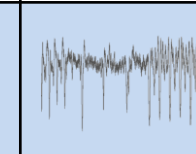
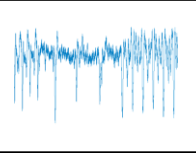
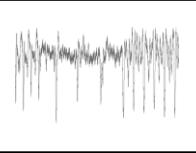
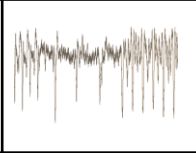
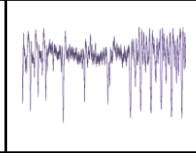
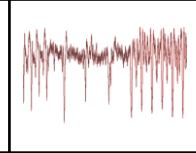
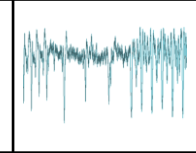
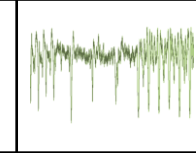
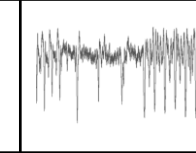


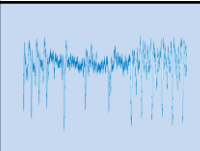
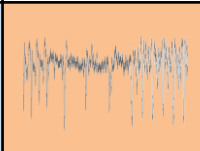
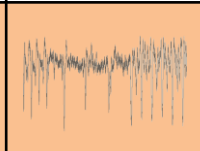
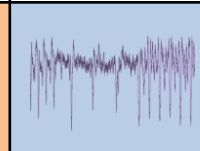
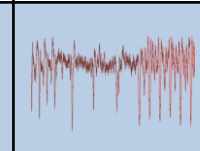
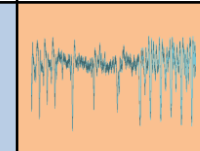
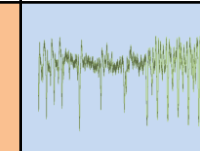
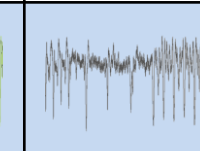
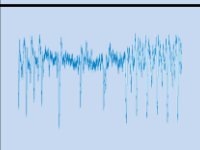
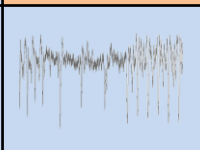
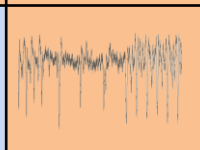
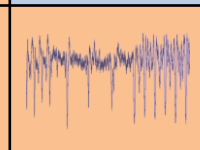
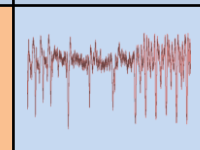
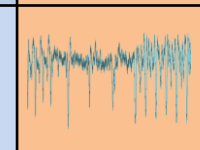
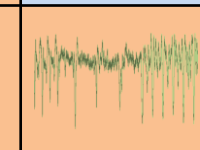
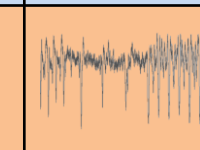
Putting traces into two groups

Example: Guess for first key byte: $k = 0x00$

Trace	0	1	2	3	4	5	...
Plaintext x	0x70	0x22	0x0C	0xFF	0x01	0x37	...
Output S-Box B_0	0x51	0x93	0xFE	0x16	0x7C	0x9A	...
MSBit(B_0)	0	1	1	0	0	1	...

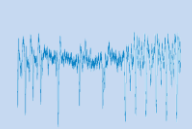
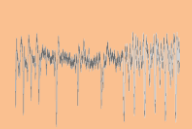
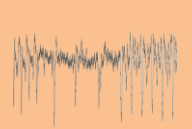
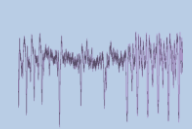
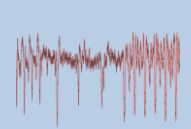
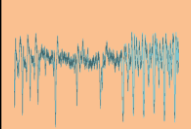
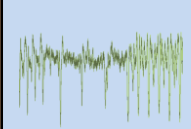
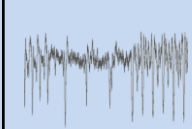
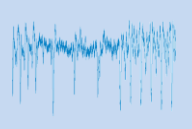
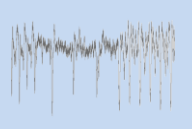
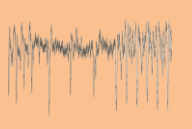
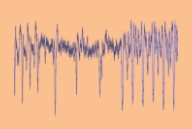
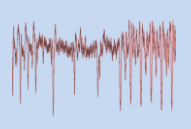
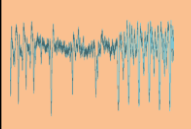
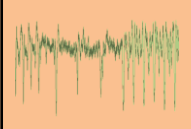
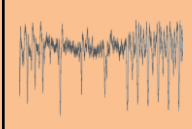
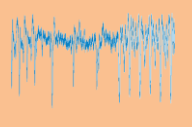
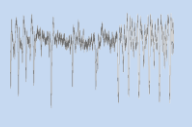
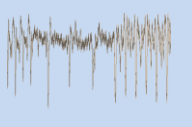
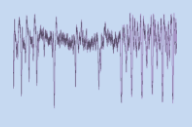
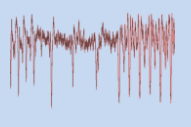
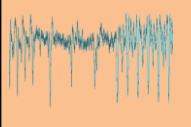
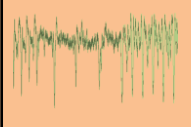
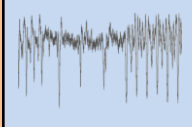
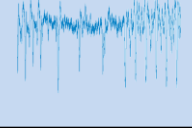
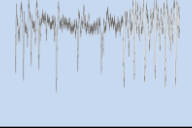
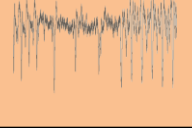
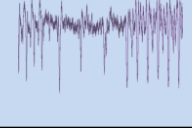
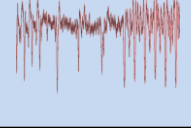
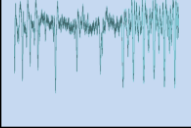
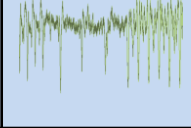
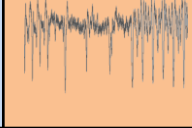


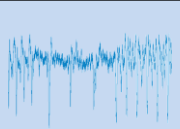
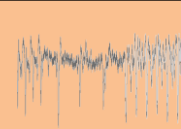
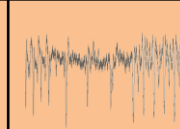
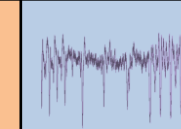
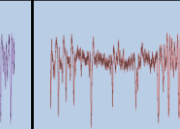
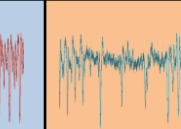
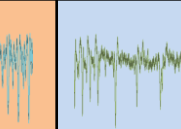
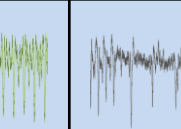
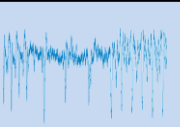
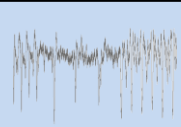
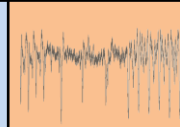
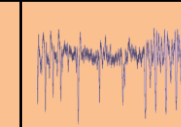
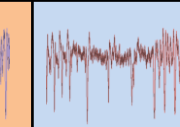
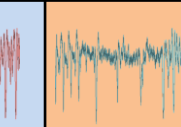
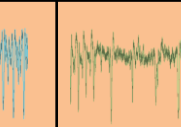
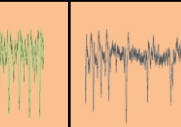
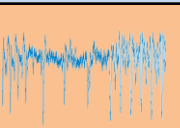
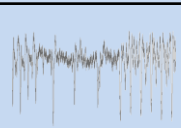
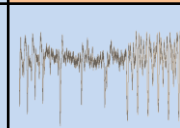
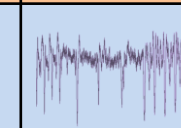
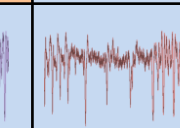
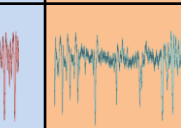
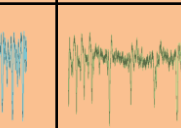
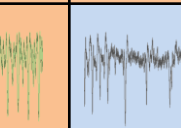
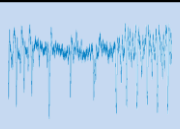
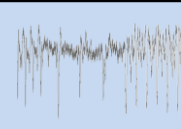
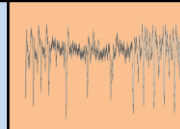
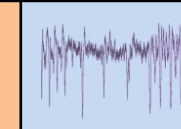
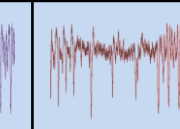
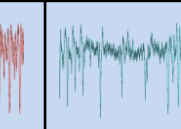
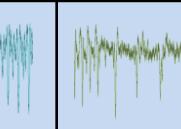
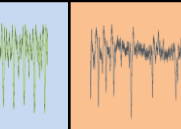
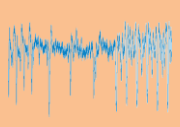
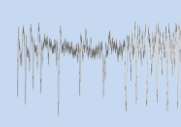
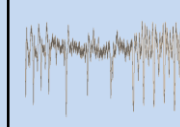
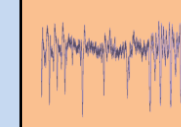
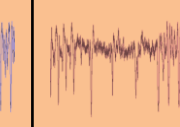
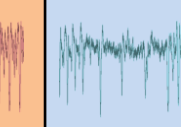
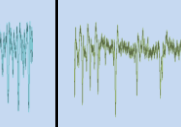
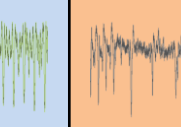
	Trace 0	1	2	3	4	5	6	7	...
k = 0x00									...
k = 0x01									...

	Trace 0	1	2	3	4	5	6	7	...
k = 0x00									...
k = 0x01									...

[illegible]

[illegible]

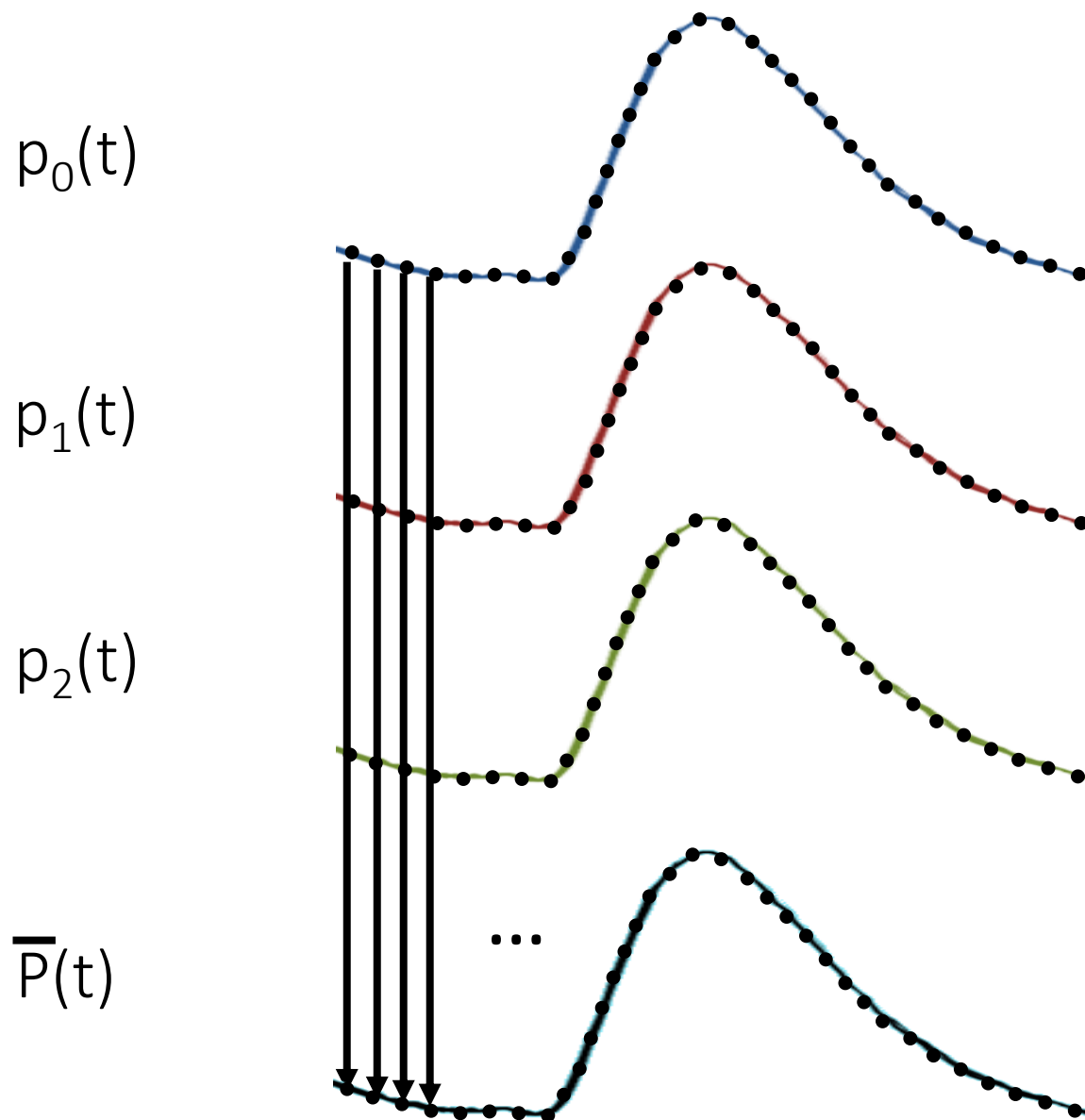
	Trace 0	1	2	3	4	5	6	7	...
k = 0x00									...
k = 0x01									...
k = 0x02									...
...	...	...	...	...	...	...	...	...	...
k = 0x12									...

	Trace 0	1	2	3	4	5	6	7	...
k = 0x00									...
k = 0x01									...
k = 0x02									...
...	...	...	...	...	...	...	...	...	...
k = 0x12									...
...	...	...	...	...	...	...	...	...	...
k = 0xFF									...

The attack ...

- For each candidate k
 - For each x
 - Compute hypothetical $\text{MSbit}(B_0)$
 - If $\text{MSBit}(B_0) == 0$: put $p(t)$ into P_0
 - Else $\text{MSBit}(B_0) == 1$: put $p(t)$ into P_1
 - Compute pointwise averages $\overline{P}_0(t)$ and $\overline{P}_1(t)$ over P_0 and P_1
 - Compute $\Delta(t; k) = \overline{P}_1(t) - \overline{P}_0(t)$
- Find $\max. \Delta(k; t_0) \rightarrow$ Recover one key byte k

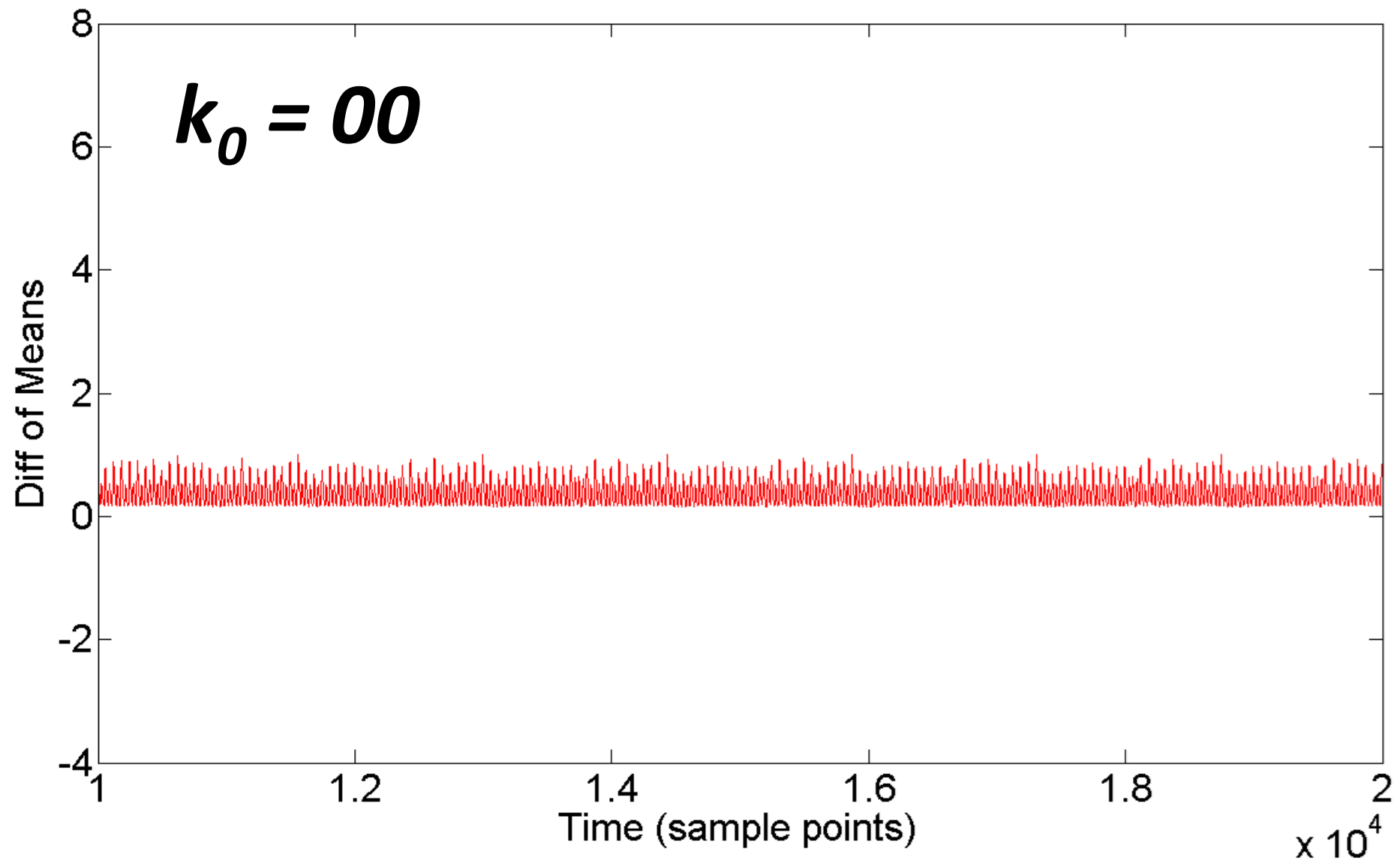
Pointwise averages



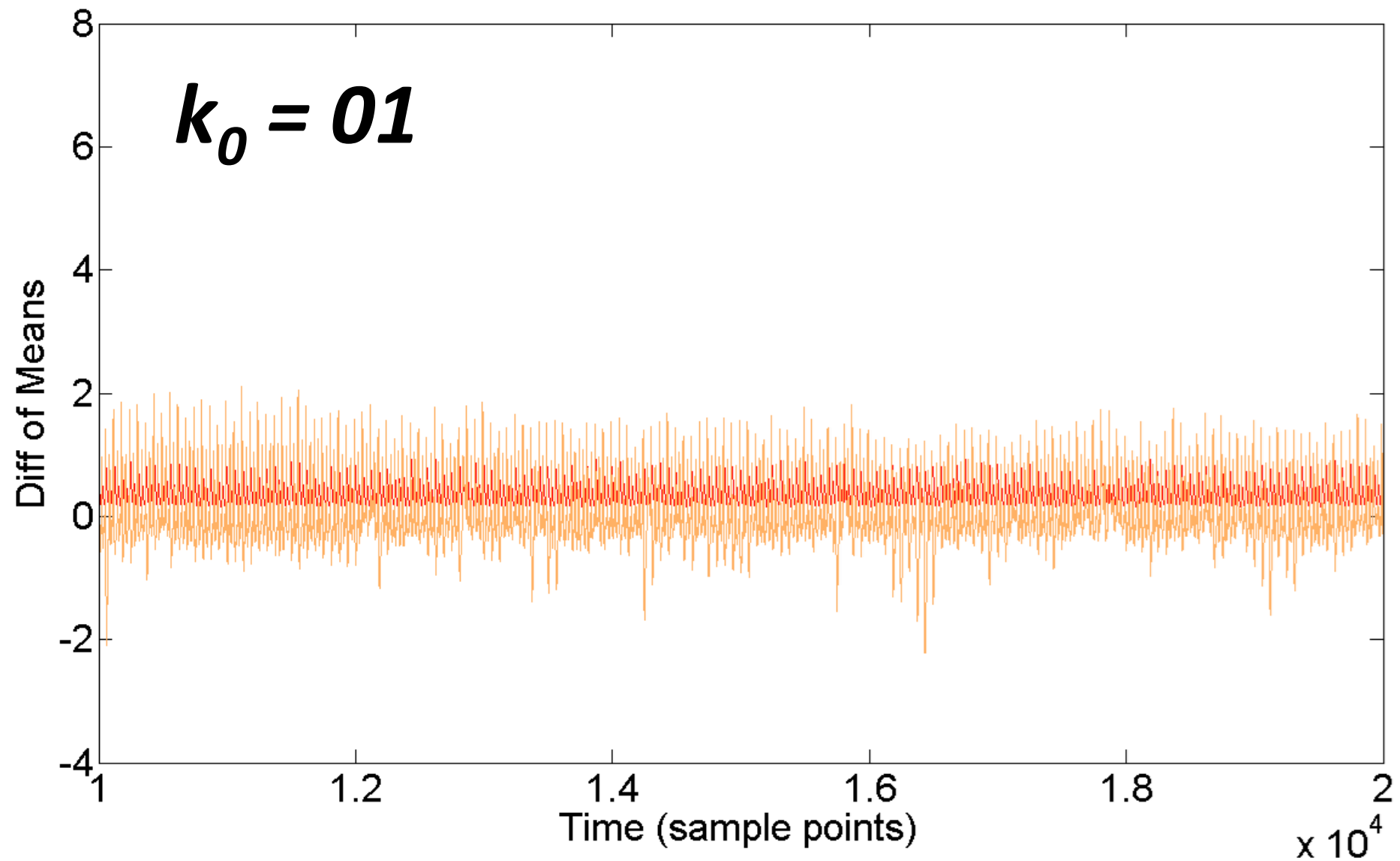
The attack ...

- For each candidate k
 - For each x
 - Compute hypothetical $\text{MSbit}(B_0)$
 - If $\text{MSBit}(B_0) == 0$: put $p(t)$ into P_0
 - Else $\text{MSBit}(B_0) == 1$: put $p(t)$ into P_1
 - Compute pointwise averages $\overline{P}_0(t)$ and $\overline{P}_1(t)$ over P_0 and P_1
 - Compute $\Delta(t; k) = \overline{P}_1(t) - \overline{P}_0(t)$
- Find $\max. \Delta(k; t_0) \rightarrow$ Recover one key byte k

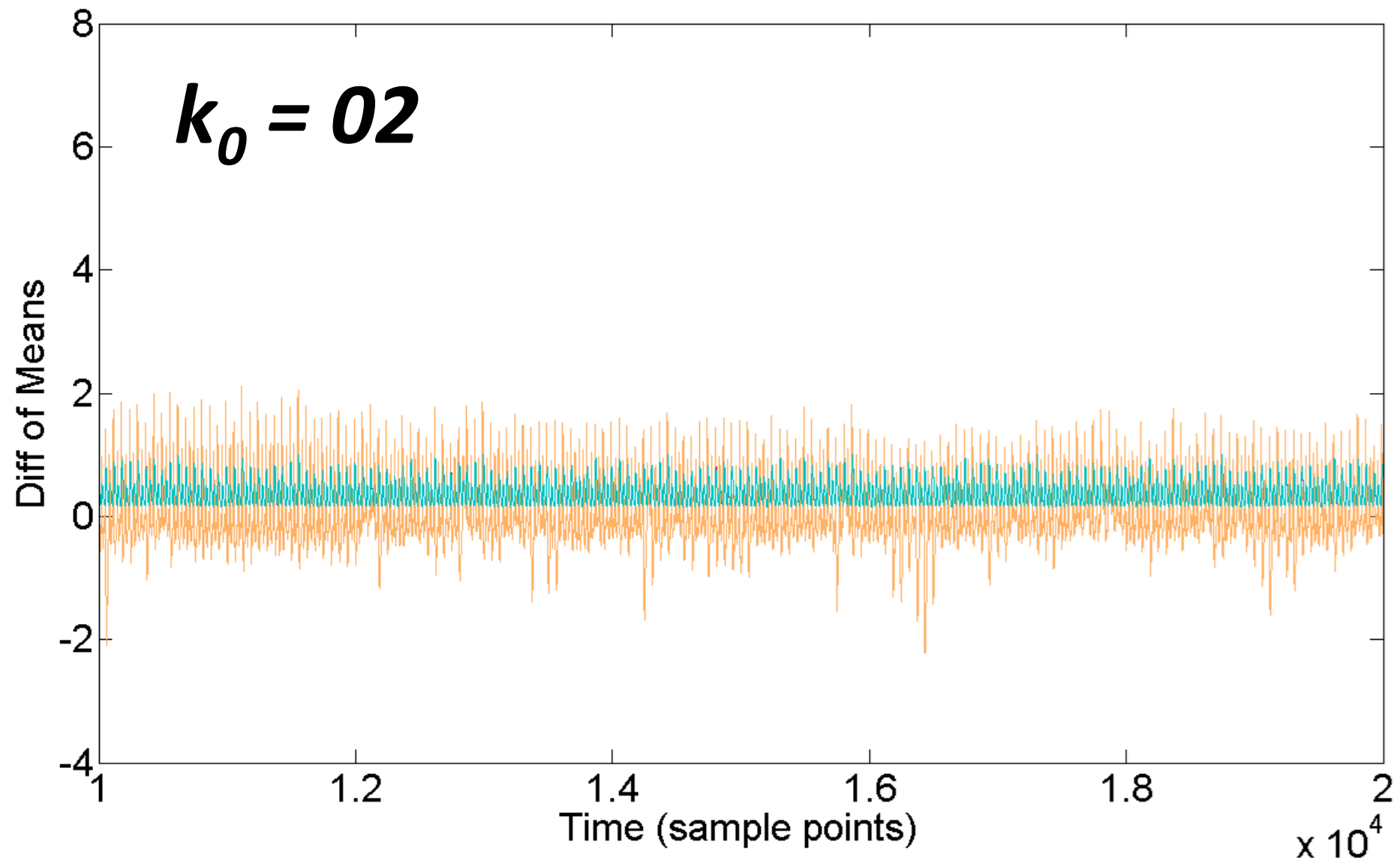
Difference of Means



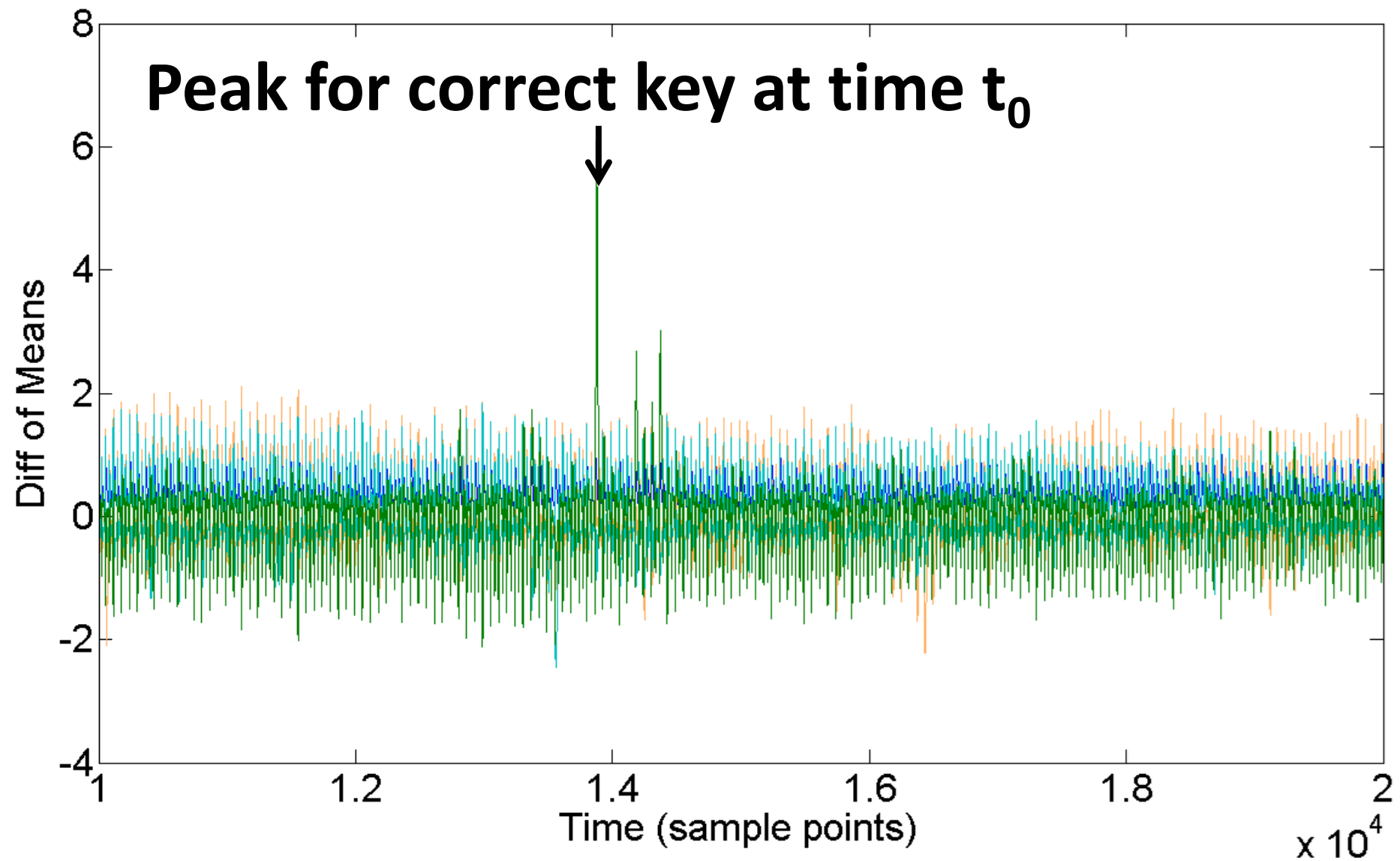
Difference of Means



Difference of Means



Difference of Means



Why does this work?

$$p(k, x, t_0) = p_{\text{sbox}}(k, x) + p_{\text{rest}}$$

$$\sum P_{\text{sbox},i}(k, x) \quad \text{noise}$$

For the **correct** k , we predict one term in the sum correctly, so the averages are:

$$\overline{P}_0 = \text{"0"} + \text{noise} + p_{\text{rest}}$$

$$\overline{P}_1 = \text{"1"} + \text{noise} + p_{\text{rest}}$$

$\Delta \rightarrow \text{const}$ for many traces

For all **wrong** k , we predict just noise:

$$\overline{P}_0 = \text{noise} + p_{\text{rest}}$$

$$\overline{P}_1 = \text{noise} + p_{\text{rest}}$$

$\Delta \rightarrow 0$ for many traces

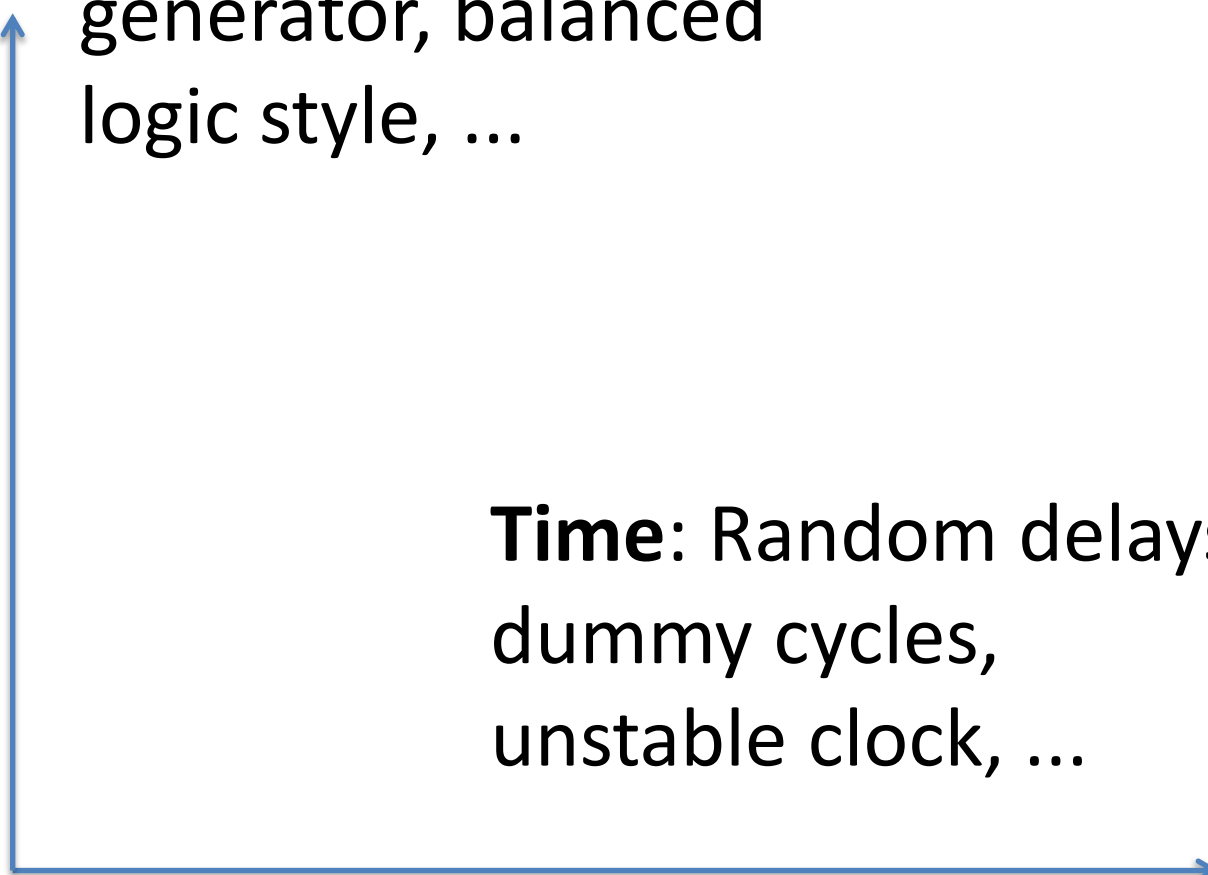
A few remarks

- We only looked at one bit (MSBit) here for simplicity – but you can really use **any**
- The “height” of the peak depends on the **difference** in power consumption for a 0/1
- The differential power analysis attack works for **most** ciphers in a similar way, and can use other statistics

Countermeasures?



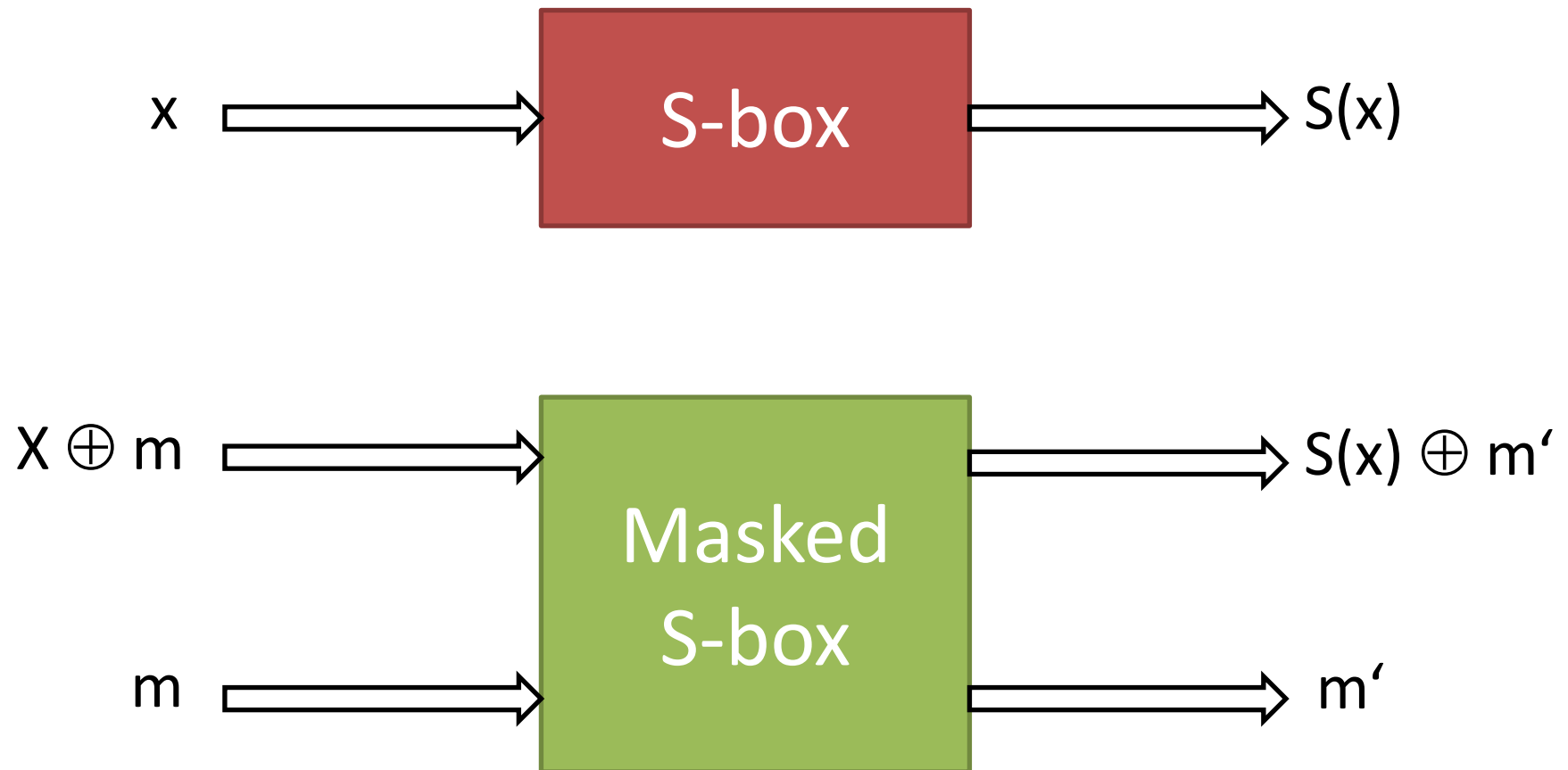
„Classical“ Countermeasures: Hiding



Amplitude: Noise generator, balanced logic style, ...

Time: Random delays, dummy cycles, unstable clock, ...

„Classical“ Countermeasures: Masking



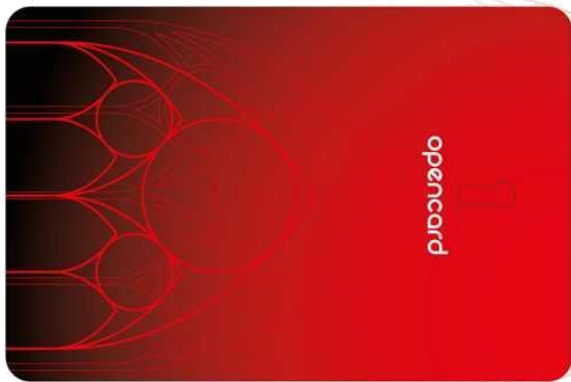
Limitations

- Substantial area and speed overhead
- Requires good randomness (often difficult on embedded devices)
- Partially depends on algorithm
- Can be attacked with higher-order DPA / CPA (look at more than one point at a time)

Countermeasures

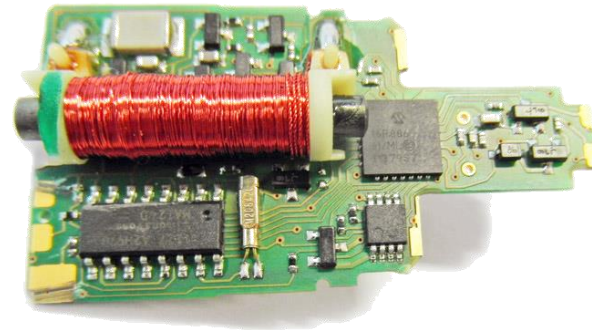
- **First line of defense:** Classical countermeasures
 - Secure hardware (certified devices)
 - Algorithmic level
- **Second line of defense:** System level
 - **Detect:** Shadow accounts, logging
 - **Minimize impact:** Key diversification

SCA: Different Scenarios



Opencard / DESFire

- Time per key: 7 h
 - Diversified keys
 - Each card: new attack
- Attack does not scale



Locking system

- Time per key: 15 min
 - Installation-wide key
 - Attack one lock
- Attack scales

Take-home points

- **Differential Power Analysis** can detect and exploit tiny differences in side-channel traces
- The idea is similar to the timing attack from the previous lecture
- There are various **extensions** to make the attack more powerful
- **Countermeasures** usually come with a cost – check your threat model!

Thanks!

Questions / discussion?

d.f.oswald@bham.ac.uk